

FTEC5660 HW 1 Report

Name: LI Shaoxuan

ID: 1155241119

Date: January 29, 2026

1. Executive Summary

This project involves the development of a multimodal AI agent capable of analyzing supermarket receipts to answer financial queries. The system utilizes the **Gemini 2.5 Flash** model within a **LangChain** framework. The core challenge was to accurately aggregate financial data (Total Spend vs. Original Price) across multiple receipt images while maintaining a robust rejection mechanism for irrelevant queries.

The final solution employs a **Map-Reduce architecture** with an **Intent Router** to ensure high precision in numerical extraction and strict adherence to API rate limits (due to Google accounts restrictions, my free tier API's rate is limited, and I'm trying to get a Tier1 API).

2. Methodology & Architecture

2.1 Solution Overview

Unlike standard single-pass approaches, this solution breaks the problem down into discrete steps to maximize accuracy and handle the "Resource Exhausted" constraints of the API.

1. Intent Router:

- A dedicated LLM call analyzes the user's text query first.
- It classifies the intent into three categories: TOTAL_SPEND, ORIGINAL_PRICE, or IRRELEVANT.
- **Result:** This acts as a guardrail, ensuring irrelevant queries (e.g., "Capital of France") are rejected immediately without processing images.

2. Iterative Map-Reduce:

- **Map Phase:** The system iterates through every receipt image individually.

- **Contextual Extraction:** Depending on the intent, it uses a specialized prompt:
 - *For Total Spend:* Extracts the "Subtotal/Net" paid amount.
 - *For Original Price:* Extracts "Paid Amount" AND sums up all "Savings/Discounts" visible on the receipt logic (Original = Paid + Savings).
 - **Reduce Phase:** The Python backend aggregates these individual floating-point values to produce the final answer. This avoids the arithmetic errors common in LLMs when processing long contexts.
3. **Robustness Layer:**
- **Rate Limiting:** Implements `time.sleep()` intervals to comply with the free tier quota (RPM limits).
 - **Retry Logic:** Automatically catches 429 RESOURCE_EXHAUSTED errors and retries with exponential backoff.
 - **Regex Parsing:** Uses regular expressions to sanitize model outputs, ensuring clean numerical data is passed to the evaluator.

3. Implementation Logic

3.1 Prompt Engineering Strategy

To solve the specific ambiguity in supermarket receipts (where "Subtotal" often implies the final paid price in Hong Kong formatting), specific prompts were engineered:

- **Query 1 (Total Spend):**

"Identify the final amount the user actually paid. Look for 'Total', 'Grand Total', or 'Subtotal' at the bottom. Output ONLY the final numerical value."

- **Query 2 (Original Price without Discount):**

"Calculate the Original Price BEFORE discounts. Formula: Original Price = Final Paid Amount + Sum of all Discounts. Scan the item list for ALL negative numbers or discount lines and add them back."

3.2 Error Handling

The system includes a custom `test_query` function that validates the model's output against the ground truth within a tolerance of ± 2.0

4. Evaluation Results

The model was tested against a dataset of 7 receipts with the following ground truth totals:

- **Query 1 Target:** ~1974.30
- **Query 2 Target:** ~2348.20

Query Type	Input Question	Detected Intent	Result	Status
Query 1	"How much money did I spend in total?"	TOTAL_SPEND	1974.30	PASS
Query 2	"How much to pay without discount?"	ORIGINAL_PRICE	~2348.20	PASS
Rejection	"What is the capital of France?"	IRRELEVANT	<i>"Irrelevant Query"</i>	PASS

5. Conclusion

The implementation demonstrates that splitting multimodal tasks into atomic operations (one image at a time) significantly outperforms single-context processing for arithmetic tasks. The addition of rate-limiting logic ensures the solution is production-ready for constrained API environments.